Computational Sociology

Supervised Machine Learning and Prediction

Dr. Thomas Davidson

University of North Carolina at Chapel Hill

October 9, 2026

Plan

1. Course updates
2. Prediction and explanation
3. Model evaluation
4. Classification algorithms
5. Machine learning in R

Course updates

Homework 2

- Homework 2 due Wednesday, October 14
- No class next week (Fall Break, October 16)

Part I: Prediction and explanation

Introduction to machine learning

What is machine learning?

- Machine learning is a method to "automate discovery from data" (Molina and Garip 2019)
- Combining statistics and computer science
- Artificial intelligence

Introduction to machine learning

Two cultures of statistical modeling

- Consider the following linear model:

$$\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

- $\hat{Y}$ is a predicted outcome
- $\hat{\beta}_0$ and $\hat{\beta}_1$ are estimated *coefficients*.
  - $\hat{\beta}_0$ is the *intercept*.
  - $\hat{\beta}_1$ is the change in the predicted outcome for a one-unit increase in x.

Introduction to machine learning

Prediction vs. explanation

- Explanatory models use theory to select variables and interpret relationships between them, focusing on the $\hat{\beta}$'s
- Predictive models aim to predict Y accurately in new data, often using many variables (or "features")
- What can social scientists learn from models optimized for prediction?

Breiman, Leo. 2001. "Statistical Modeling: The Two Cultures." *Statistical Science* 16(3): 199–231.

Introduction to machine learning

Supervised and unsupervised learning

- *Supervised learning*: learn to predict observed outcomes Y from features X.
  - Classification predicts categories. Regression predicts continuous outcomes.
- *Unsupervised learning*: find patterns in X without an outcome label.
  - Topic modeling, PCA, embedding.

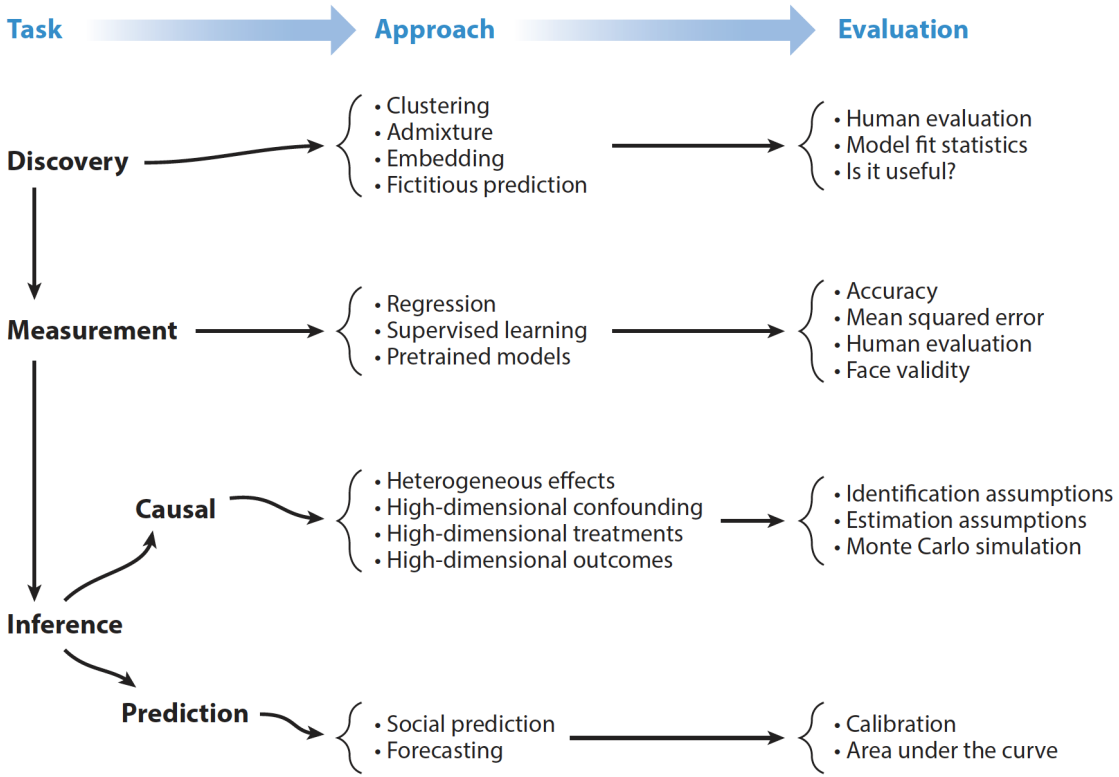
Introduction to machine learning

Why predict?

- Policy: identify people who may need an intervention.
- Document classification: identify relevant texts and classify their content.
- Record linkage and imputation: match records and estimate missing values.
- Causal inference: estimate quantities such as propensity scores and explore differences in treatment effects.

Introduction to machine learning

Social science applications



Grimmer, Roberts, and Stewart 2021

Introduction to machine learning

Predictive validity

- Social scientists often evaluate explanations through coefficients and significance tests, with less attention to prediction.
- Watts (2014) argues that theories should also be evaluated by how well they predict observed outcomes.
- What good is an explanation if it isn't predictive?

Watts 2014. See also Turco and Zuckerman 2017; Watts 2017.

Introduction to machine learning

Prediction and explanation in social systems

- Hofman, Sharma, and Watts identify three priorities:
  1. Use consistent practices to evaluate predictions.
  2. Understand the limits to prediction in social systems.
  3. Evaluate explanations through both predictive accuracy and interpretability.

Hofman, Sharma, and Watts 2017

Introduction to machine learning

Data splitting and model training

- *Training data*: learn a function $\hat{Y} = f(X)$ from observed outcomes.
- *Validation data*: compare models and choose settings.
- *Test data*: evaluate the final model on observations not used to fit or select it.
  - Compare predictions $\hat{Y}$ with observed outcomes Y.

Introduction to machine learning

Example: Explanatory modeling

- What explains whether someone voted for Trump in 2020?
- Specify a regression model with K predictors selected using theory:

$$Y_{trump} = \beta_0 + \sum_{k=1}^{K} \beta_k X_k + \epsilon$$

- Interpret the estimated associations, for example:
  - "Respondents who attend religious services weekly were 23 percentage points more likely to vote for Trump ($p < 0.001$)."

Linear probability model controlling for age, sex, race, and degree. 2024 GSS, unweighted, N = 2,447.

Introduction to machine learning

Example: Predictive modeling

- How accurately can we predict whether someone voted for Trump?
- Fit a model using the training data, including more predictors and a more flexible functional form.
- Predict vote choice in the test data and compare predictions with observed votes.
- Report performance, for example:
  - "The model predicted vote choice with 85% accuracy on the test data."

Introduction to machine learning

Explanatory models $\neq$ predictive models

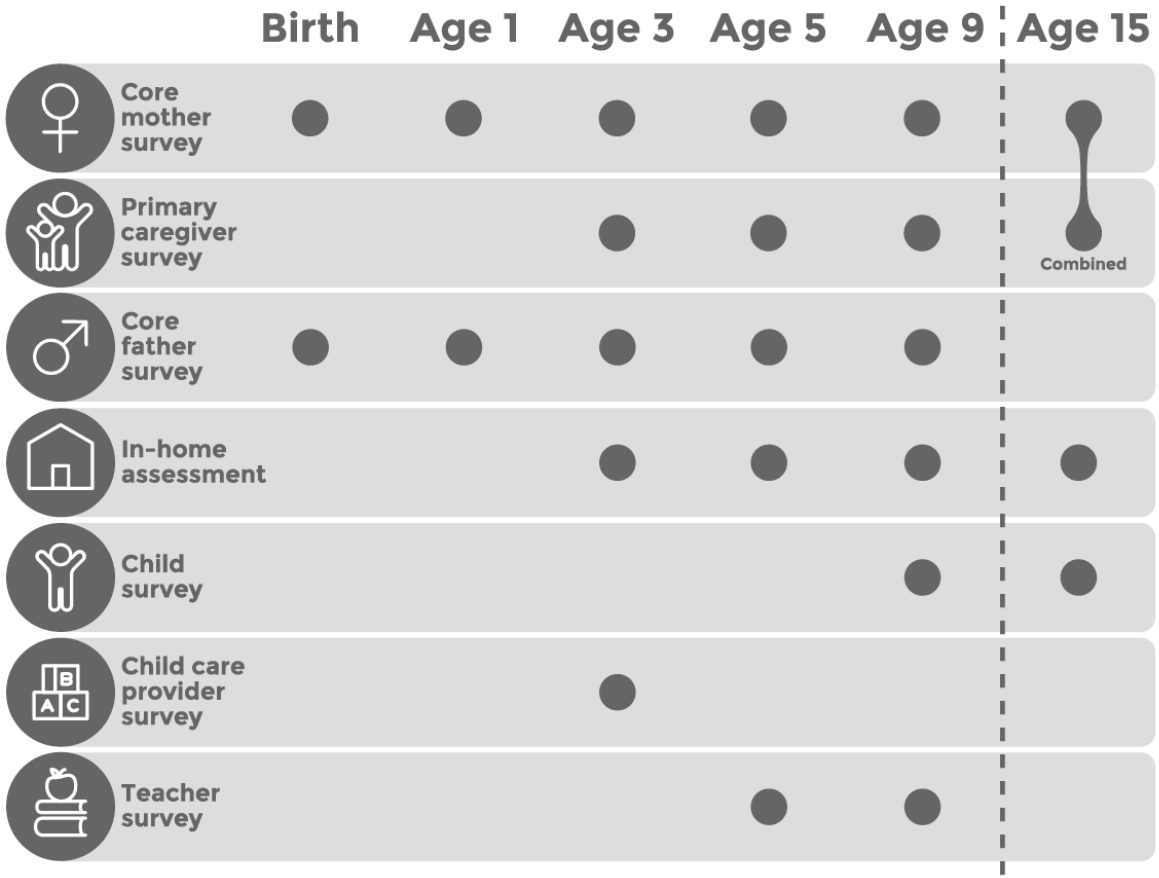
- Can we have models that are good at prediction and explanation?

"The very appeal of these algorithms is that they can fit many different functions. But this creates an Achilles' heel: more functions mean a greater chance that two functions with very different coefficients can produce similar prediction quality"

See Mullainathan, Sendhil, and Jann Spiess. 2017. "Machine Learning: An Applied Econometric Approach." Journal of Economic Perspectives 31 (2): 87–106.

The (un)predictability of social life

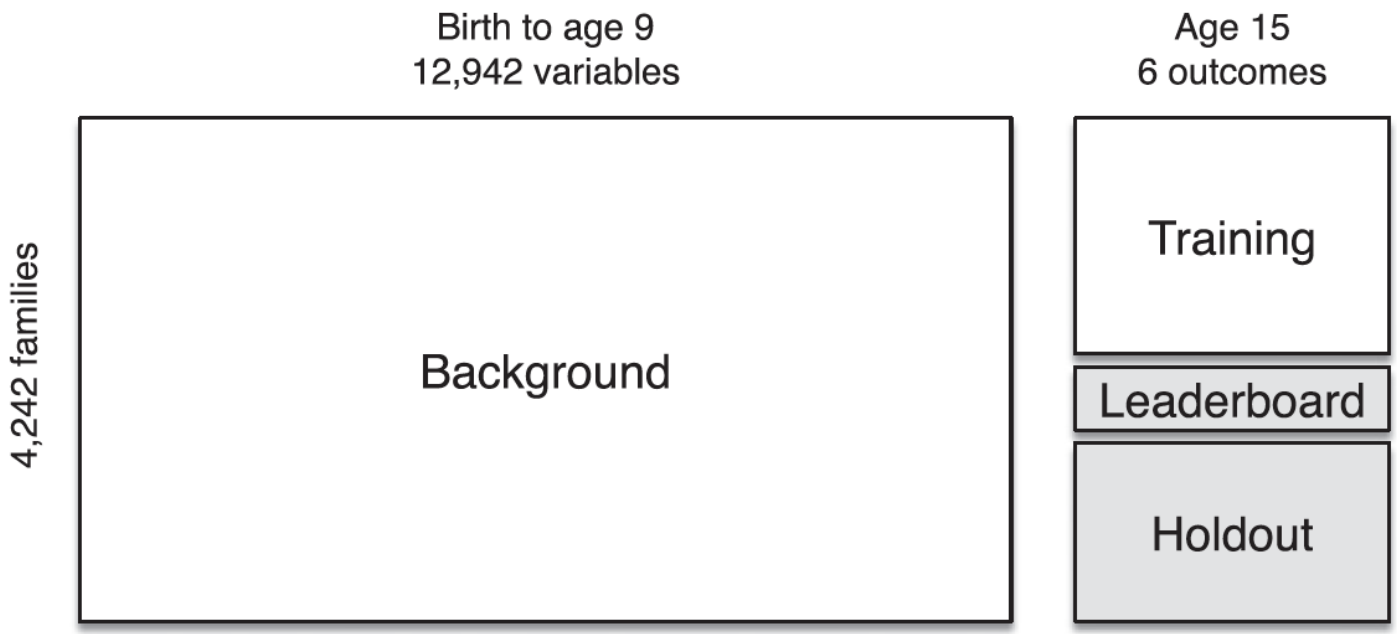
The Fragile Families Challenge



Salganik et al. 2020

The (un)predictability of social life

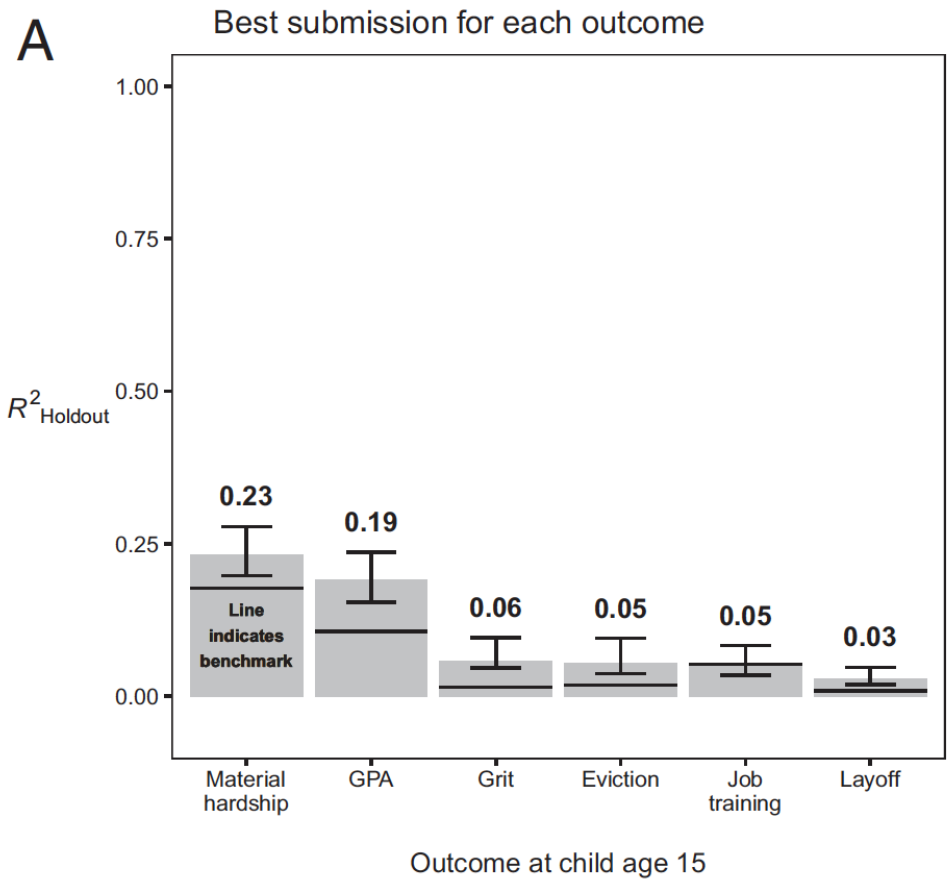
The Fragile Families Challenge



Salganik et al. 2020

The (un)predictability of social life

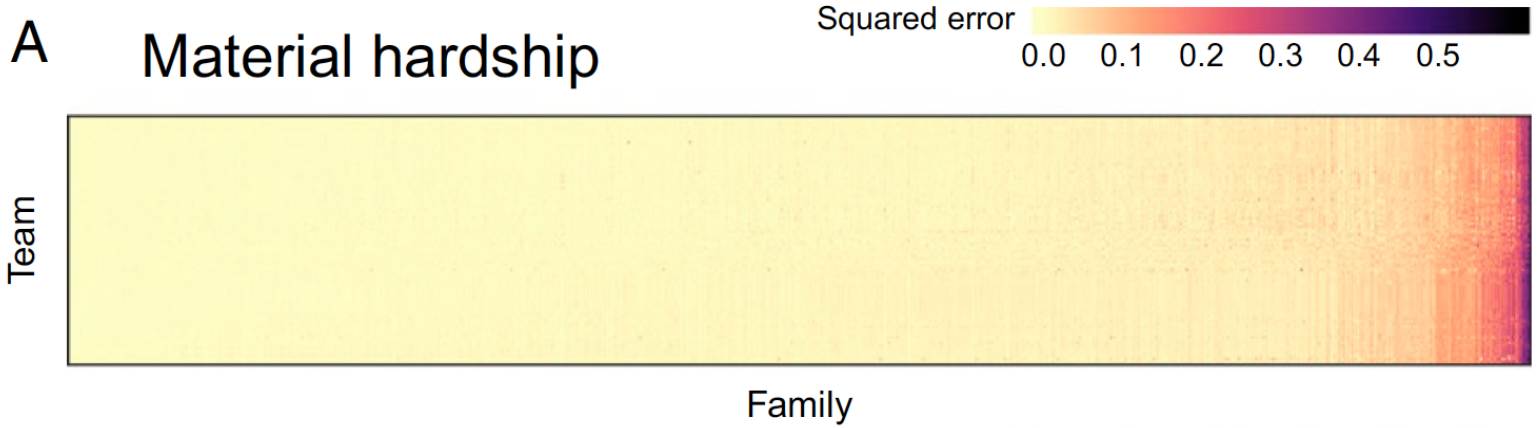
The Fragile Families Challenge



Salganik et al. 2020

The (un)predictability of social life

The Fragile Families Challenge



Salganik et al. 2020

The (un)predictability of social life

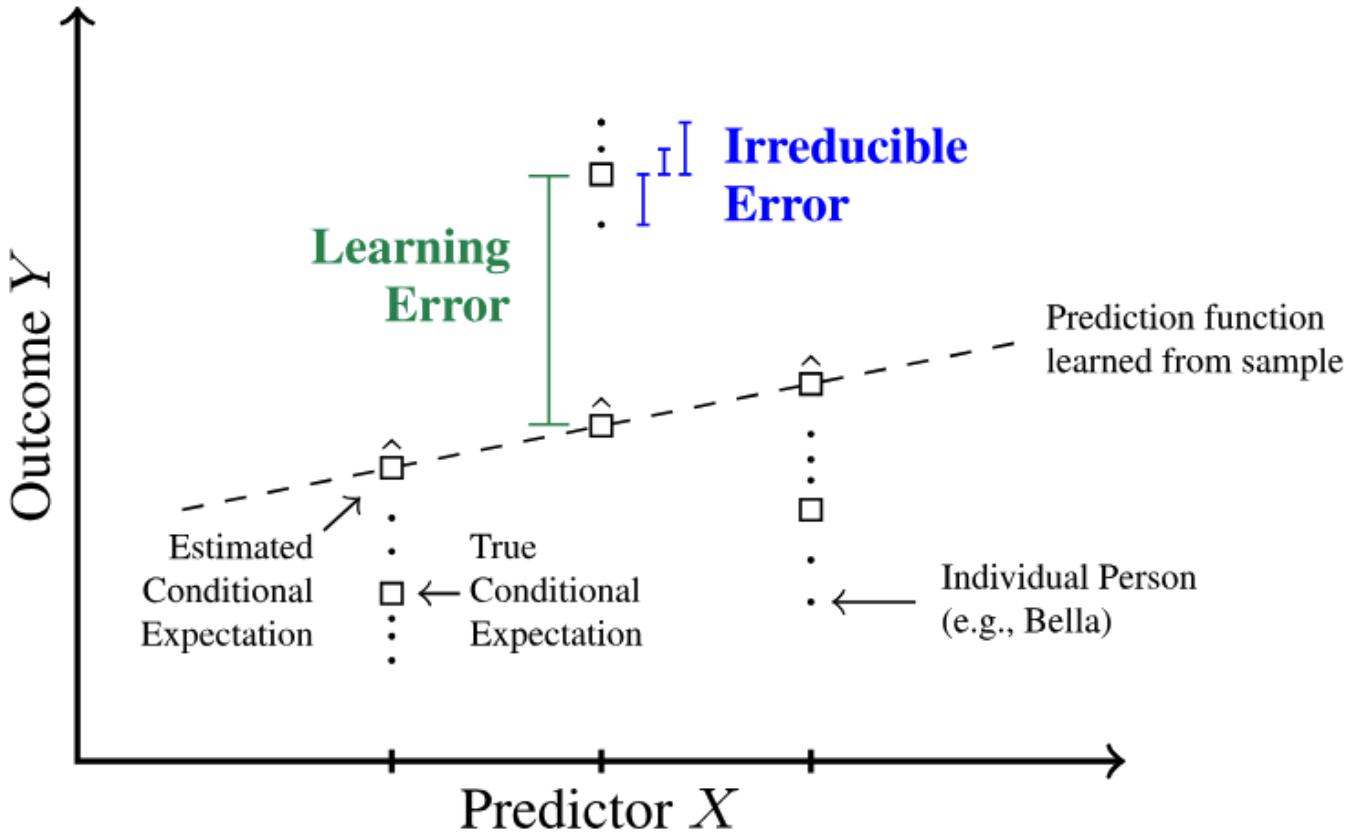
Explaining unpredictability

- Can better measurements improve our predictions, or are some outcomes inherently difficult to predict?
- Lundberg et al. (2024) interviewed 40 families from the study to investigate prediction errors. They distinguish between two sources of error:
  - *Learning error*: The degree to which predicted values are far from the conditional mean in the population.
  - *Irreducible error*: "Irreducible error exists to the degree that observationally identical children have outcomes that vary."
    - Unmeasured features
    - Imperfectly measured features

Lundberg et al. 2024. "The Origins of Unpredictability in Life Outcome Prediction Tasks." *PNAS* 121(24): e2322973121.

The (un)predictability of social life

Explaining unpredictability



Part II: Model evaluation

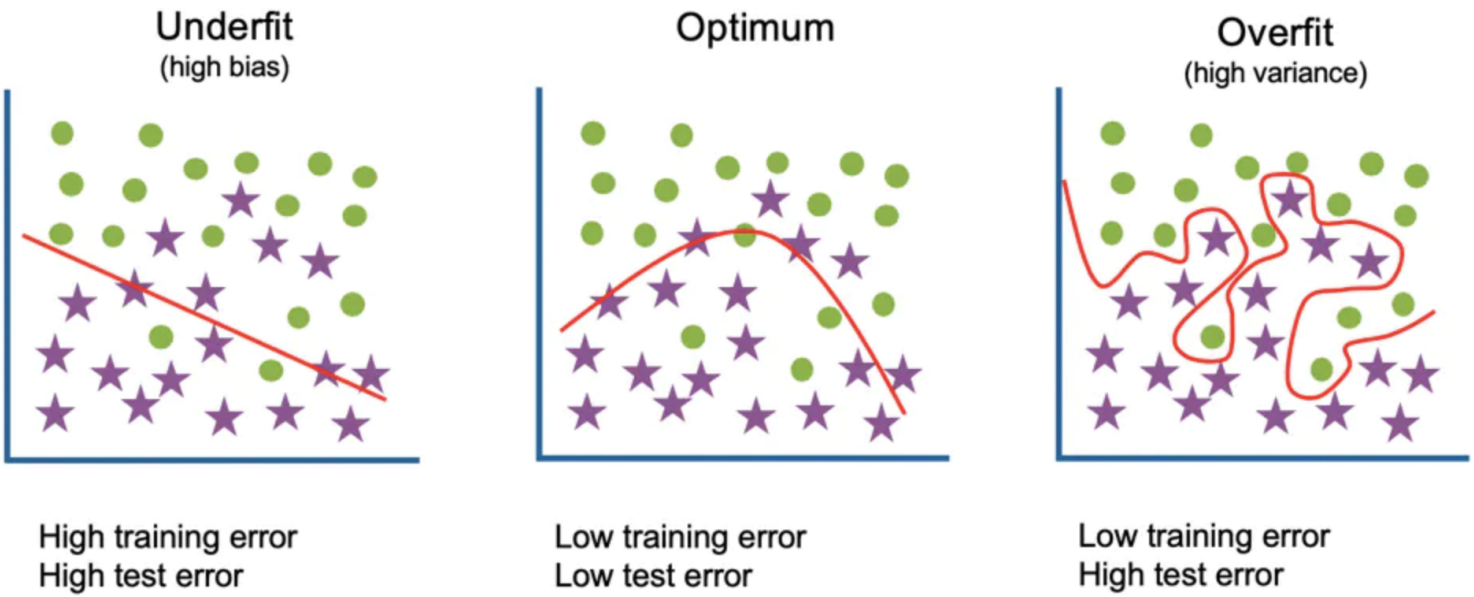
Model evaluation

Underfitting and overfitting

- *Underfitting*: the model misses patterns in the data.
  - A linear model may miss a nonlinear relationship.
  - Performance is poor on both training and test data.
- *Overfitting*: the model learns noise or details specific to the training data.
  - Performance is much better on training data than on new observations.

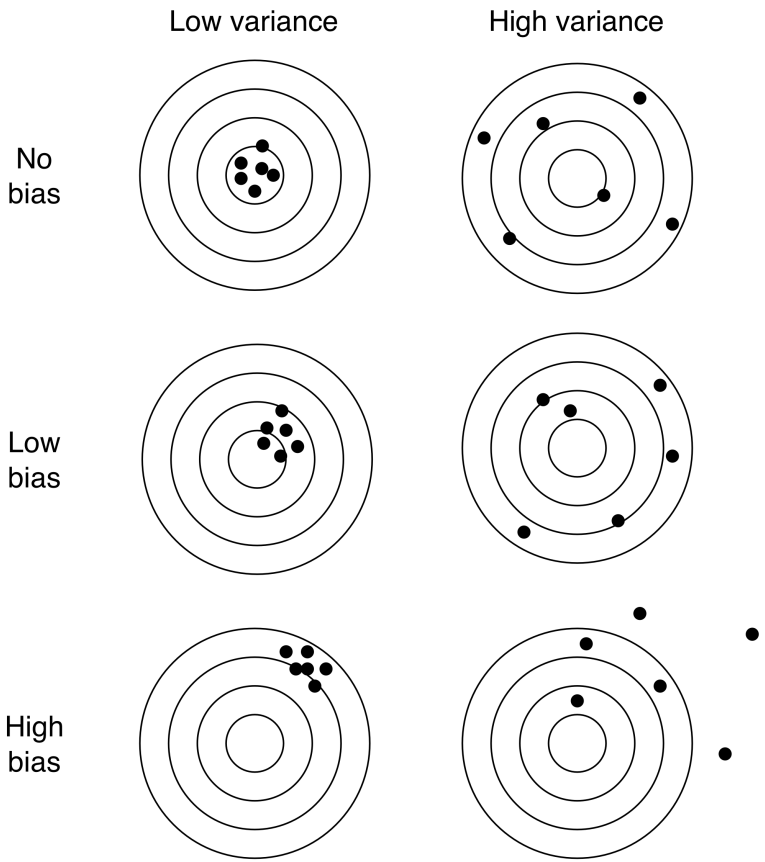
Model evaluation

Underfitting and overfitting



Model evaluation

Bias-variance trade-offs



Salganik 2017. Interactive explanation: http://www.r2d3.us/visual-intro-to-machine-learning-part-2/

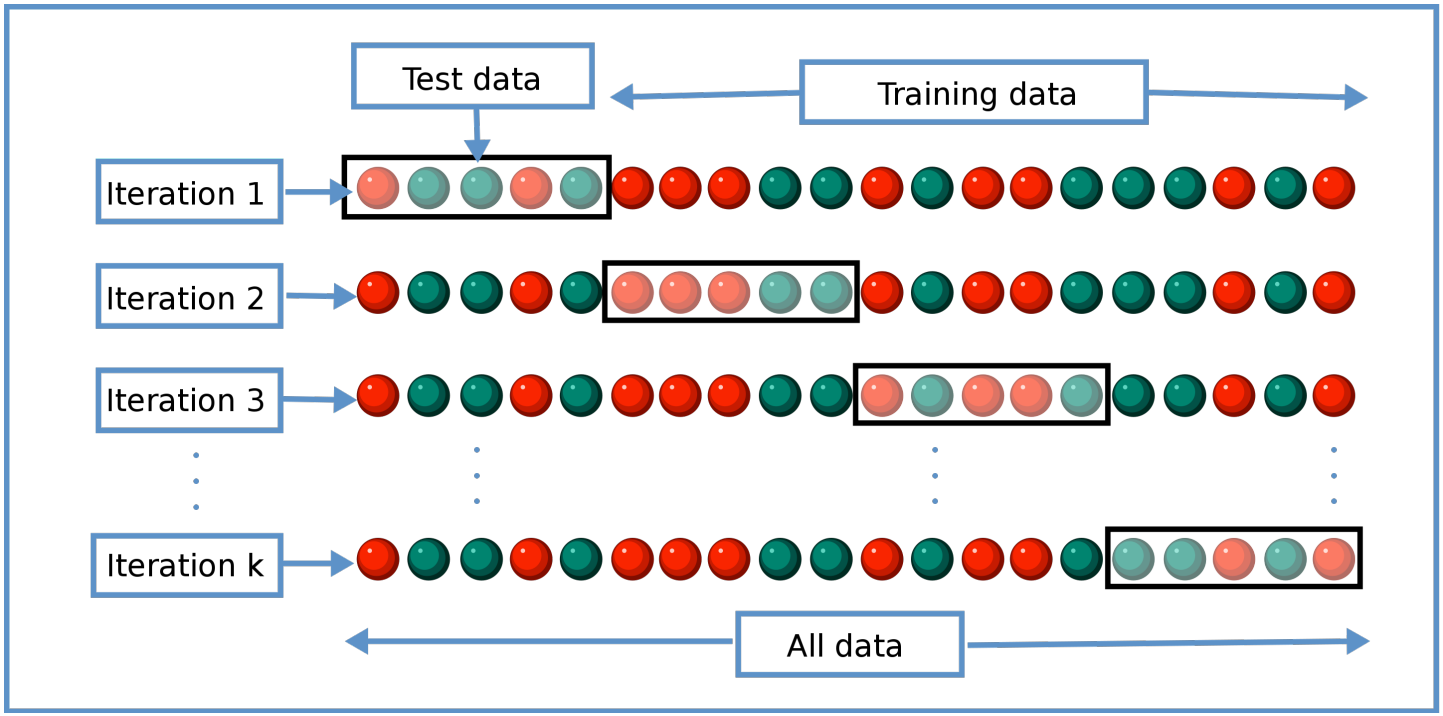
Model evaluation

Cross-validation

- Results from one train-test split can depend on which observations fall in each group.
- In *k-fold cross-validation*, divide the training data into k folds.
  - Train on $k - 1$ folds and evaluate on the remaining fold.
  - Repeat for each fold and average performance.
- Use these results to select models; reserve the test set for final evaluation.

Model evaluation

Cross-validation



Source: Wikipedia.

Model evaluation

Regularization

- *Regularization* limits model complexity to reduce overfitting.
- LASSO (*least absolute shrinkage and selection operator*) adds a penalty to the least-squares objective:

$$\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{k} |\beta_j|$$

- λ controls the penalty's strength. Some coefficients can become zero, removing those predictors.
- Other forms include limiting tree depth and *dropout*, which randomly sets some neural network units to zero during training.

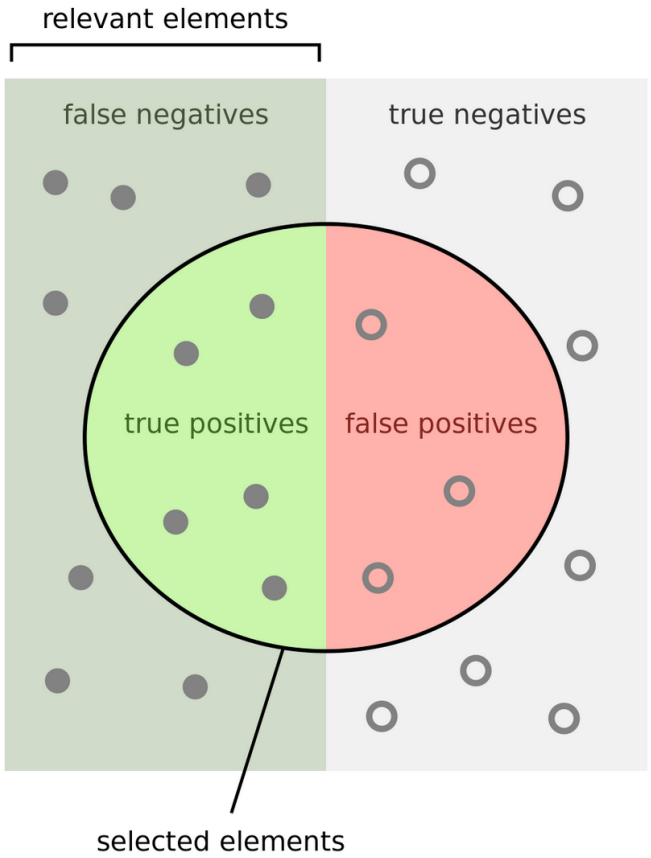
Model evaluation

Metrics: Binary classification

- A binary classifier predicts one of two classes, $Y = 1$ or $Y = 0$.
- Some models also return a predicted probability, $\hat{P}(Y = 1 \mid X)$.

Model evaluation

Metrics: TP, FP, TN, FN

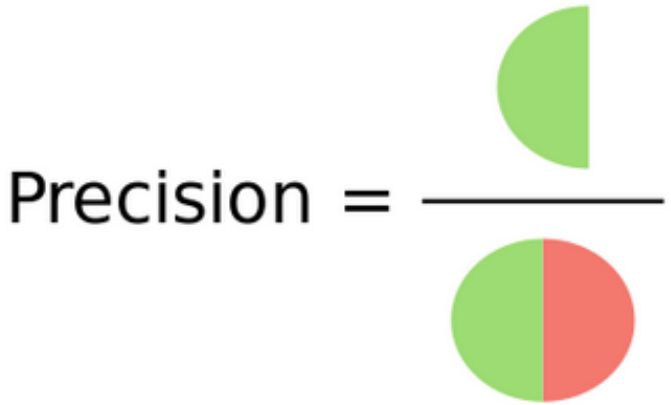


Source: Wikipedia.

Model evaluation

Metrics: Precision (TP/(TP+FP))

How many selected items are relevant?

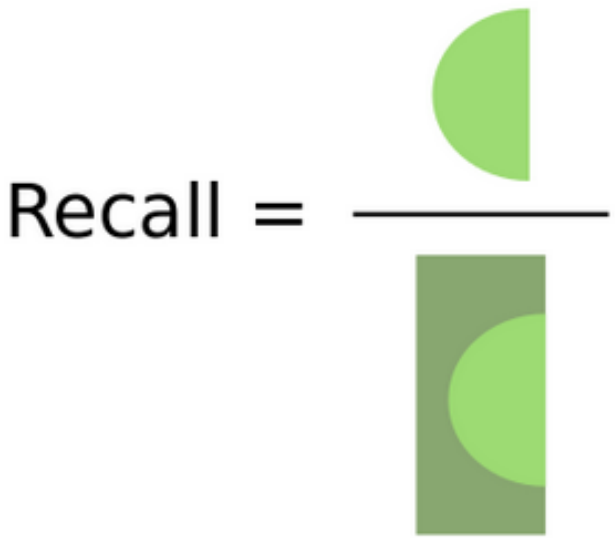


Source: Wikipedia.

Model evaluation

Metrics: Recall (TP/(TP+FN))

How many relevant items are selected?



Source: Wikipedia.

Model evaluation

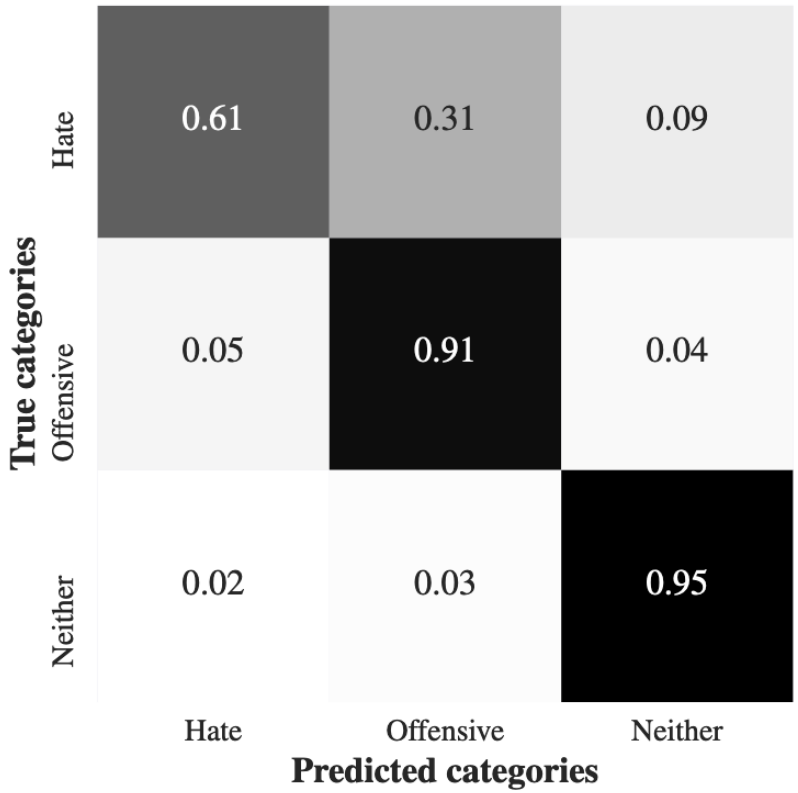
Metrics: F1

The F_1 score combines precision and recall using their *harmonic mean*.

$$F_1 = 2\frac{precision * recall}{precision + recall} = \frac{TP}{TP + \frac{1}{2}(FP + FN)}$$

Model evaluation

Metrics: The confusion matrix



Davidson, Thomas, Dana Warmsley, Michael Macy, and Ingmar Weber. 2017. "Automated Hate Speech Detection and the Problem of Offensive Language." In *Proceedings of the 11th International AAAI Conference on Web and Social Media*, 512–15.

Model evaluation

Metrics: Receiver Operating Characteristic (ROC) curve

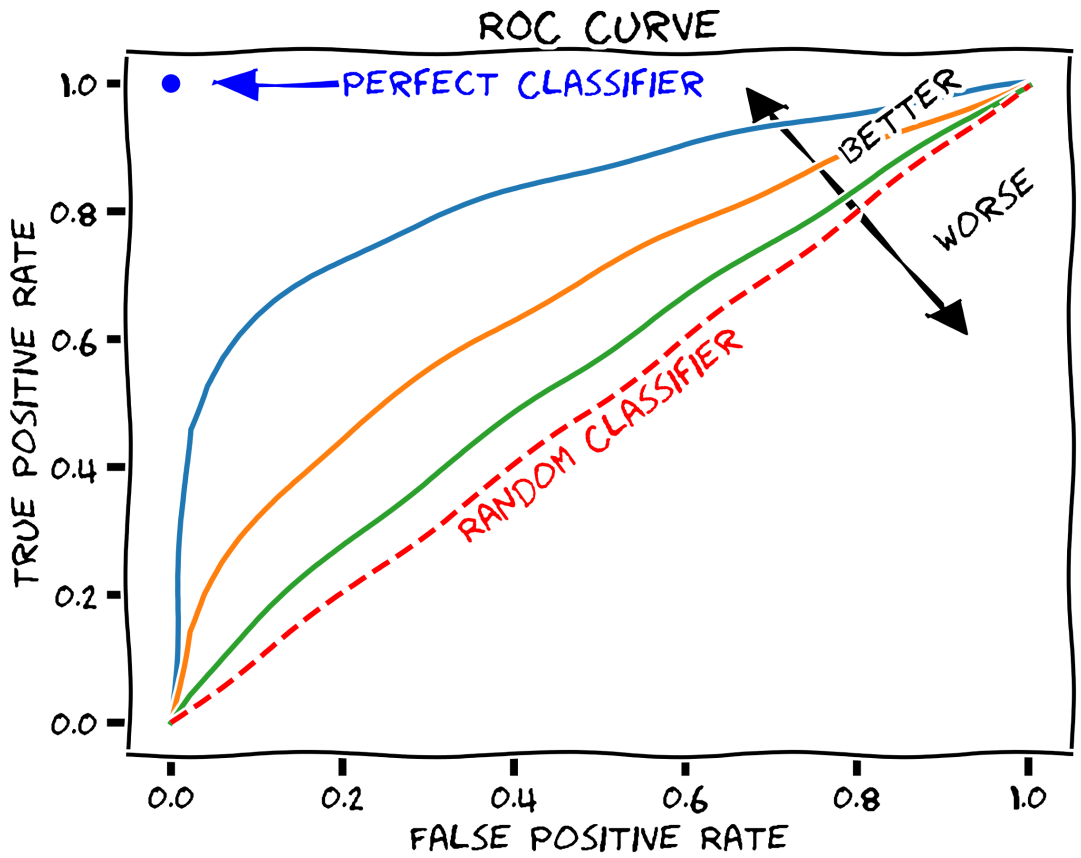
- To convert predicted probabilities into classes, choose a *threshold* t:

$$\hat{Y} = \begin{cases} 1, & \hat{P}(Y = 1 \mid X) \geq t \\ 0, & \hat{P}(Y = 1 \mid X) < t \end{cases}$$

- The *ROC curve* plots the true positive rate ($TP/(TP + FN)$) against the false positive rate ($FP/(FP + TN)$) across thresholds.
- *AUC* (area under the curve) summarizes how well the model ranks positive cases above negative cases.
- Select a threshold according to the costs of false positives and false negatives.

Model evaluation

Metrics: Receiver Operating Characteristic (ROC) curve



Source: Wikipedia

Model evaluation

Which metrics to optimize?

- Consider a carbon monoxide alarm. Which errors are worse?

	Alarm	No alarm
CO present	TP	FN
CO absent	FP	TN

Part III: Classification algorithms

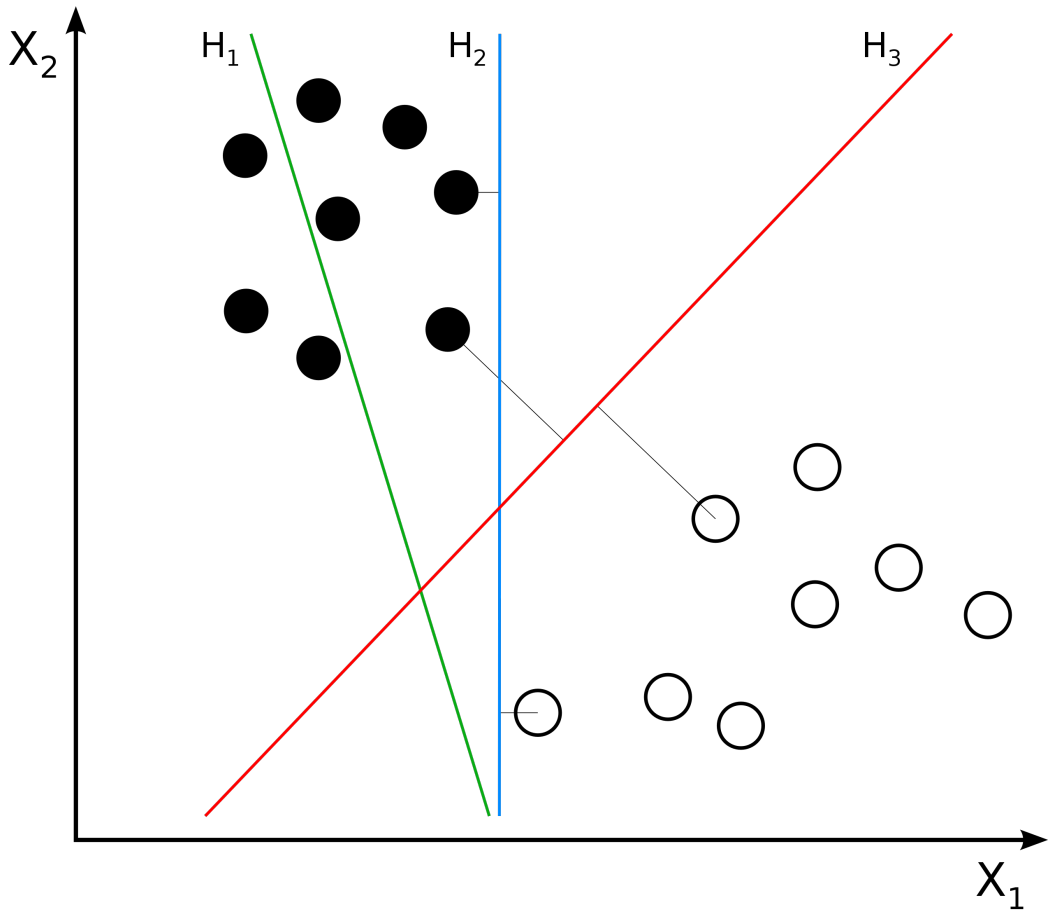
Classification algorithms*

Logistic regression

- Logistic regression models the probability of a binary outcome.
- It estimates the log-odds of $Y = 1$ as a linear function of predictors X.
- Multinomial logistic regression handles outcomes with more than two categories, known as *multiclass* classification.

Classification algorithms

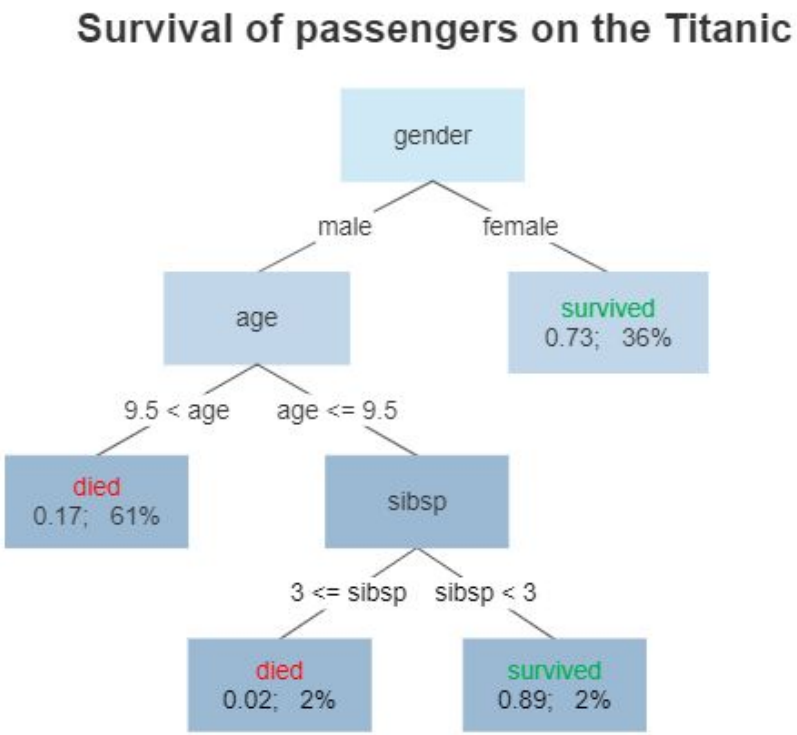
Support Vector Machines (SVM)



Source: Wikipedia

Classification algorithms

Decision Trees



Source: Wikipedia. Interactive introduction: http://www.r2d3.us/visual-intro-to-machine-learning-part-1/

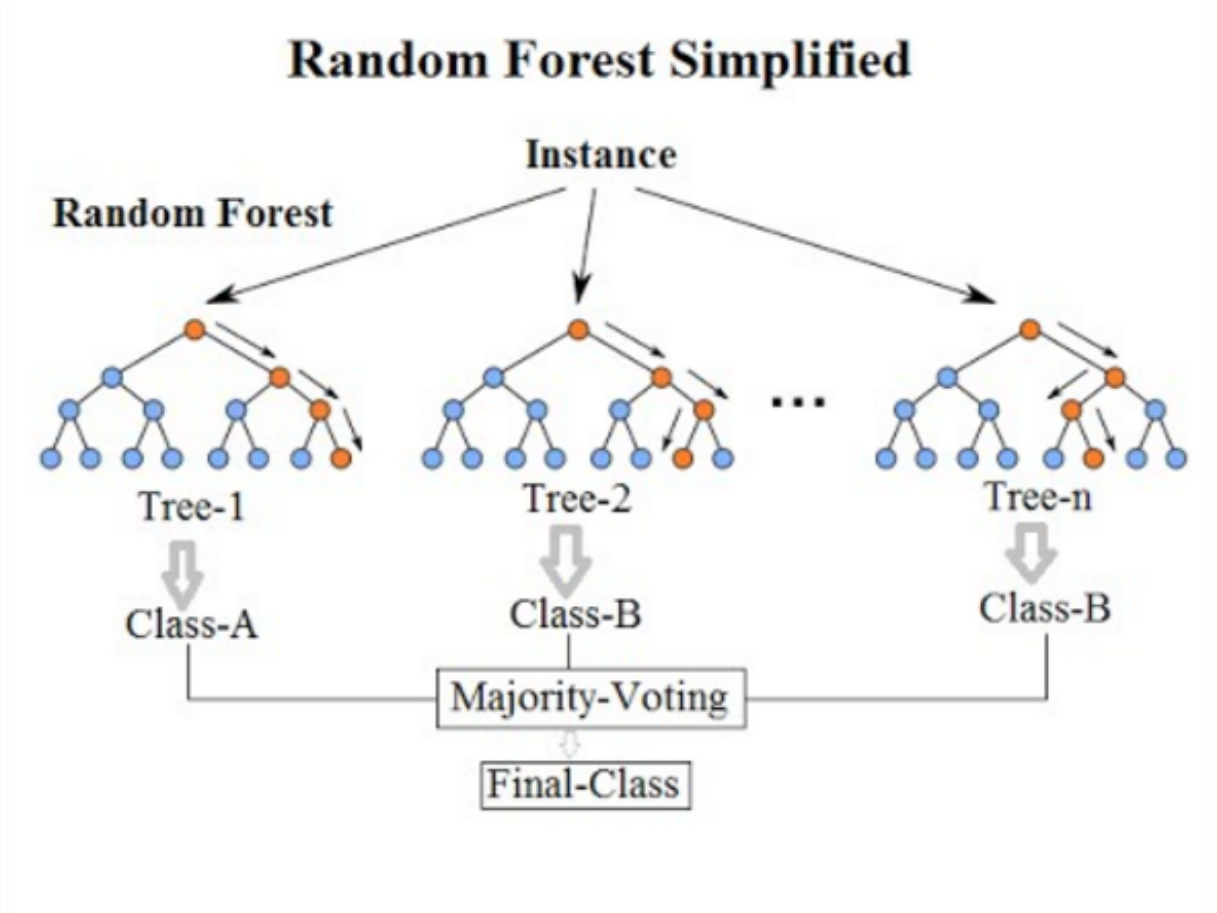
Classification algorithms

Random Forests

- Individual decision trees can easily overfit.
- A random forest combines many trees.
  - Each tree uses a bootstrap sample of observations (*bagging*) and a random subset of features at each split.
- Final classification uses the trees' majority vote.

Classification algorithms

Random Forests



Source: Wikipedia

Classification algorithms

Trees and effect heterogeneity

- *Causal trees* find subgroups with different treatment effects.
- Splits seek differences in treatment effects rather than accuracy in predicting Y.
- "Honest" estimation uses separate samples to choose the splits and estimate effects within leaves, otherwise there is a risk of overfitting and p-hacking.

Brand et al. 2021

Classification algorithms

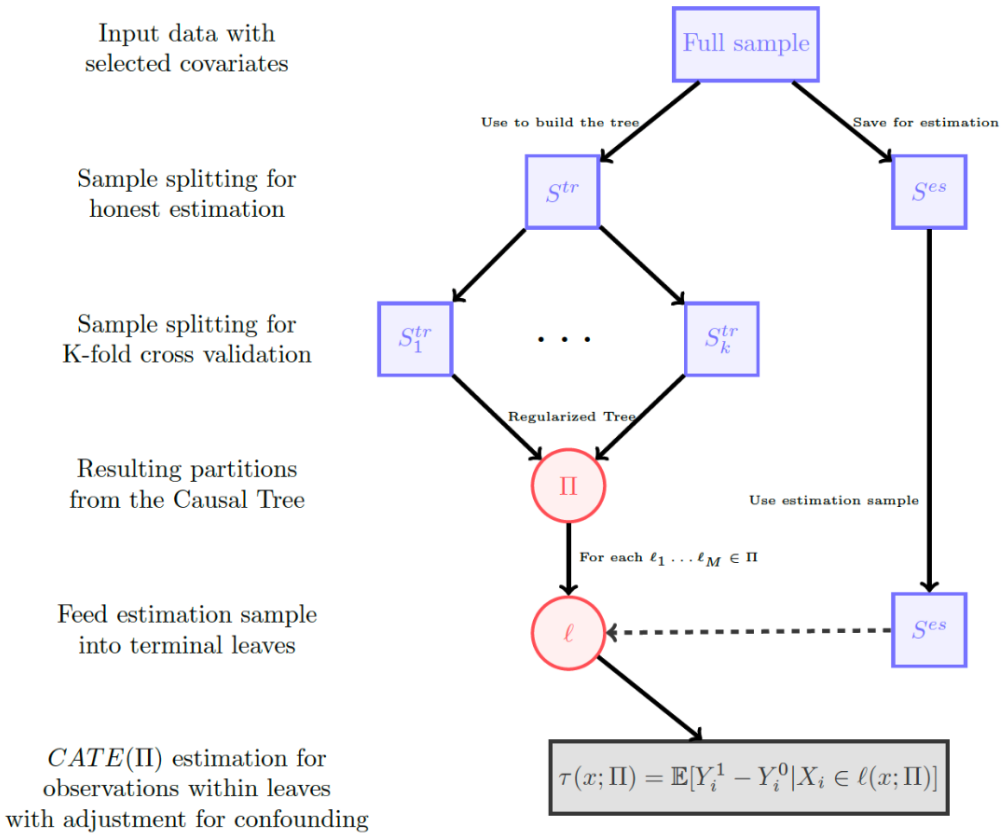
Trees and effect heterogeneity: College and low-wage work

- Brand et al. use the NLSY 1979 cohort to study college completion and time spent in low-wage work.
- After adjustment for observed confounding, college completion is associated with an 18–19 percentage point reduction in the proportion of time spent in low-wage work.
  - Causal trees used to estimate effect heterogeneity across subgroups.

Brand et al. 2021

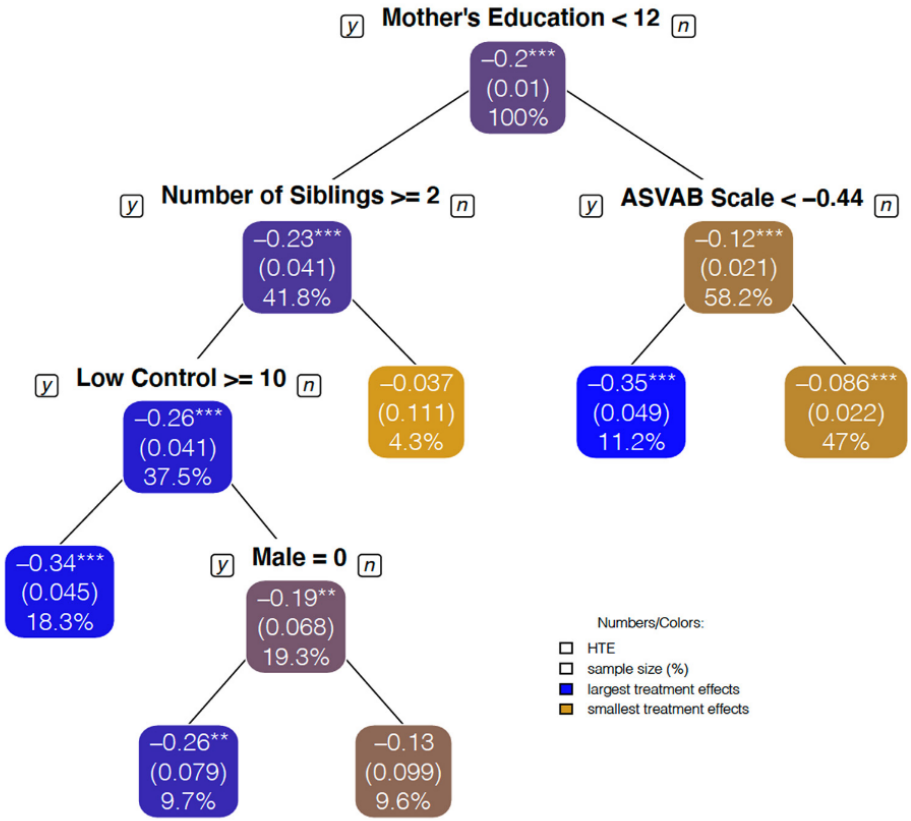
Classification algorithms

Causal trees



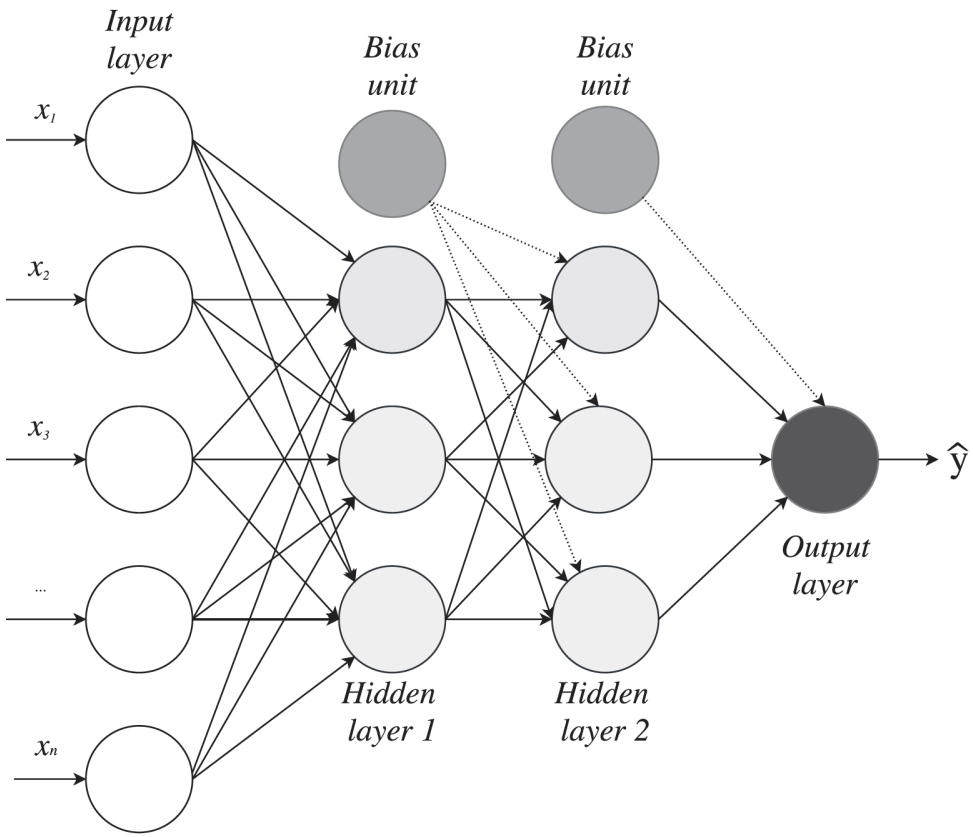
Classification algorithms

Causal trees



Classification algorithms

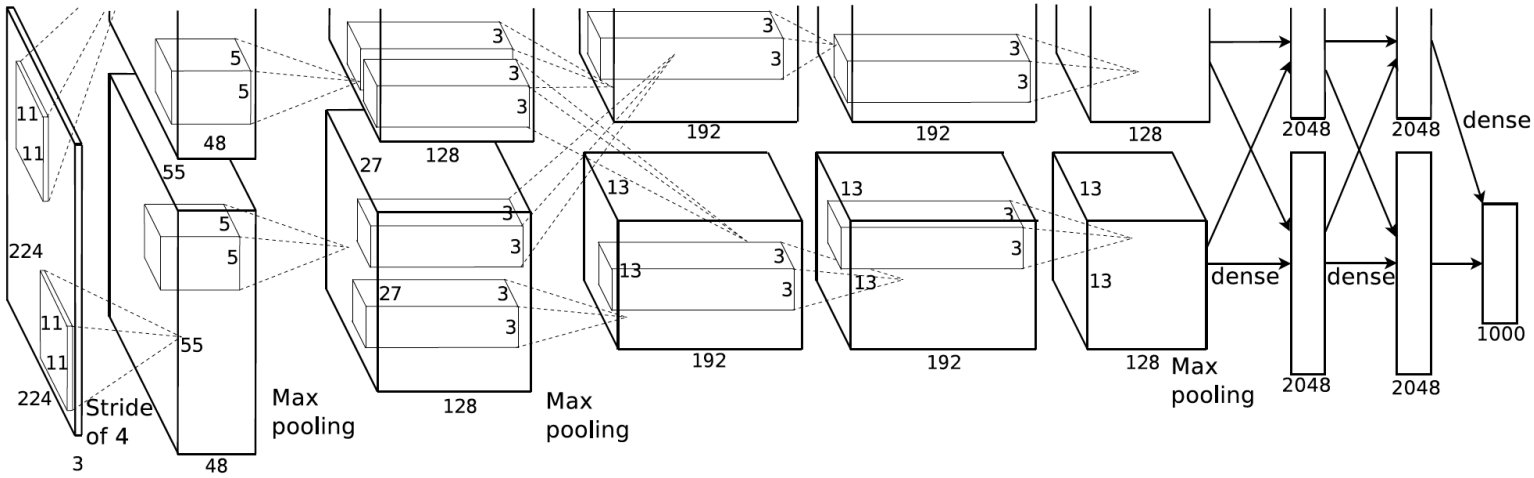
Neural networks



Davidson, Thomas. 2019. "Black-Box Models and Sociological Explanations: Predicting High School Grade Point Average Using Neural Networks." *Socius* 5.

Classification algorithms

Neural networks



Krizhevsky, Sutskever, and Hinton 2012

Classification algorithms

Hyperparameters

- Hyperparameters control how a model learns.
  - Logistic regression and SVM: regularization strength.
  - Random forest: number of trees and number of features considered at each split.
  - Neural networks: number of layers, units, and activation functions.
- Compare settings using validation data and/or cross-validation.
  - Preprocessing and feature choices can also be compared.

Classification algorithms

Hyperparameter search and computational complexity

- Davidson (2019) compared 40 neural network specifications for predicting high school GPA, varying:
  - Number of hidden layers
  - Number of units per layer
  - Activation function
- Five-fold cross-validation requires $40 \times 5 = 200$ model fits, ~12 hours of compute time.

Code and output: https://github.com/t-davidson/fragile-families-challenge/blob/master/model/gpa.ipynb

Classification algorithms

Black-box models and interpretability

- Models that combine many nonlinear relationships are "black boxes".
- Watts (2014) argues that better predictions may sometimes justify less interpretable models.
- Interpretability is a branch of machine learning focused on interpreting complex models.

Classification algorithms

Black-box models



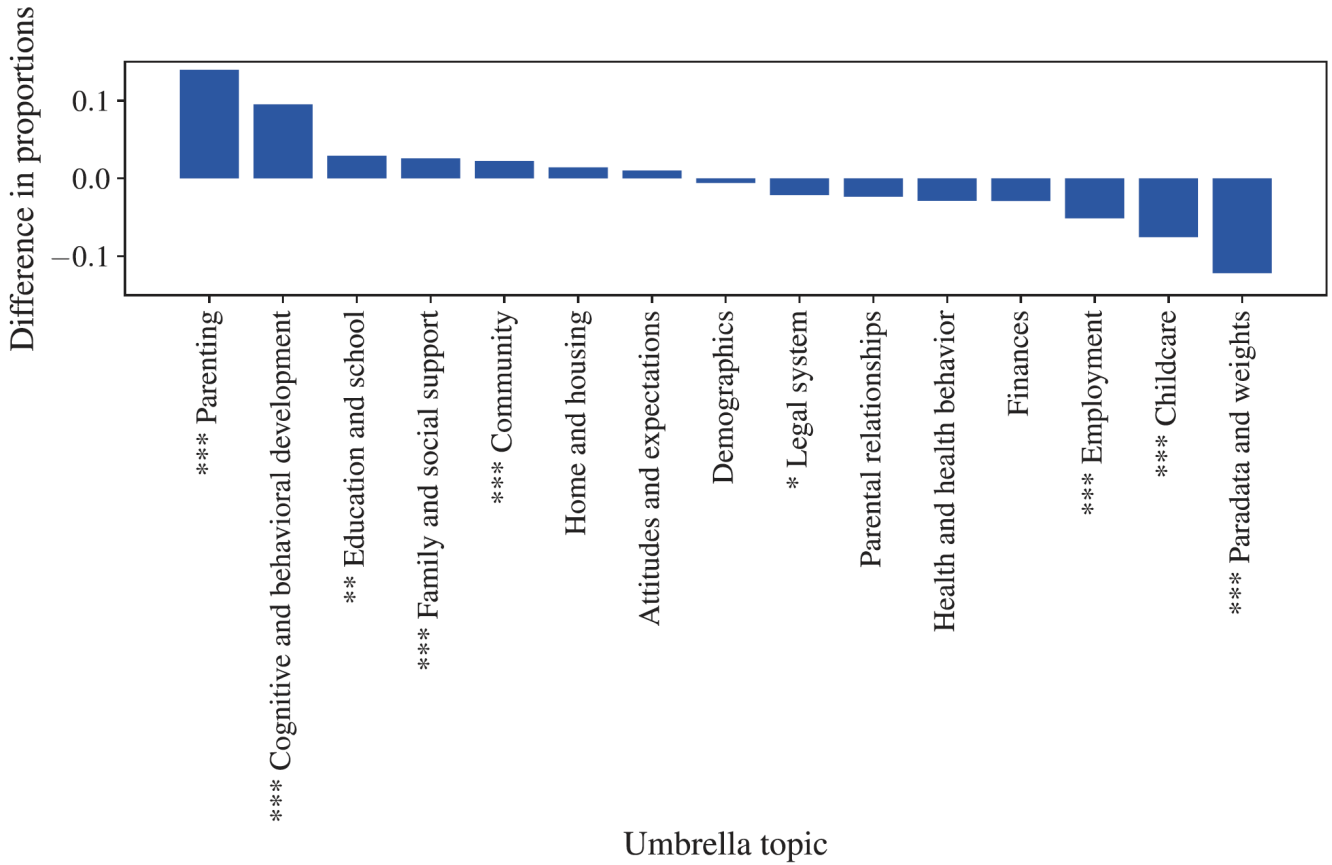
(a) Original Image (b) Explaining *Electric guitar* (c) Explaining *Acoustic guitar* (d) Explaining *Labrador*

Figure 4: Explaining an image classification prediction made by Google's Inception network, highlighting positive pixels. The top 3 classes predicted are "Electric Guitar" ($p = 0.32$)**, "Acoustic guitar"** ($p = 0.24$) **and "Labrador"** ($p = 0.21$)

Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin. 2016. " 'Why Should I Trust You?': Explaining the Predictions of Any Classifier." In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–44.

Classification algorithms

Black-box models



Davidson, Thomas. 2019. "Black-Box Models and Sociological Explanations: Predicting High School Grade Point Average Using Neural Networks." *Socius* 5.

Part IV: Machine learning in R

Machine learning in R

Tidymodels

- `tidymodels` is a collection of R packages for preparing data, fitting models, and evaluating predictions.
  - Packages: https://www.tidymodels.org/packages/

Pre-Process → Train → Validate



Source: tidymodels tutorial. https://rviews.rstudio.com/2019/06/19/a-gentle-intro-to-tidymodels/

Machine learning in R

Installing `gssr`

The `gssr` package provides General Social Survey (GSS) data. Install it from GitHub:

```
install.packages("remotes")
remotes::install_github("kjhealy/gssr")
```

Machine learning in R

Loading the 2024 GSS

```
library(tidyverse)
library(tidymodels)
library(gssr)
library(haven)

gss24 <- gss_get_yr(2024)
dim(gss24)
```

```
## [1] 3986  989
```

The 2024 file combines the main sample with an oversample of Black, Hispanic, and AAPI respondents.

Machine learning in R

The outcome: 2020 presidential vote

```
gss24 |> count(pres20 = as_factor(pres20)) |> filter(n > 0)
```

```
## # A tibble: 8 x 2
##   pres20                          n
##   <fct>                       <int>
## 1 biden                        1506
## 2 trump                         962
## 3 other candidate                61
## 4 didn't vote for president      16
## 5 iap                          1212
## 6 don't know                     46
## 7 no answer                      38
## 8 skipped on web                145
```

Machine learning in R

Selecting features

Keep respondents who reported voting for a candidate. Select social and demographic features, plus party identification.

```
data <- gss24 |>
  filter(pres20 %in% 1:3) |>
  mutate(trump = as.factor(as.integer(pres20 == 2))) |>
  select(trump, age, sex, race, hispanic, degree, region, xnorcsiz,
         marital, childs, relig, attend, born, realinc, wrkstat,
         class, partyid, pres16)
```

Machine learning in R

Data cleaning

Convert special missing values to NA and categorical variables to factors. Keep age, number of children, and income numeric. Drop incomplete observations for this example.

```
data <- data |>
  mutate(across(-trump, zap_missing)) |>
  mutate(across(c(age, childs, realinc), as.numeric)) |>
  mutate(across(-c(trump, age, childs, realinc), as_factor)) |>
  drop_na() |>
  droplevels()

table(data$trump)
```

```
##
##    0    1
## 1187  688
```

Machine learning in R

Splitting data

```
set.seed(4389)
data_split <- initial_split(data, prop = 0.8)
print(data_split)
```

```
## <Training/Testing/Total>
## <1500/375/1875>
```

Machine learning in R

Preprocessing with `recipe`

```
data_recipe <- training(data_split) |>
  recipe(trump ~ .) |>
  step_rm(partyid,pres16) |> # omit these variables
  step_dummy(all_factor_predictors()) |> # create dummies
  prep()
```

Common to convert categorical variables to dummy variables, known as "one-hot encoding"

Machine learning in R

Extracting data from `recipe`

```
data_testing <- data_recipe |>
  bake(testing(data_split))

data_training <- data_recipe |>
  bake(new_data = NULL)
```

`prep()` learns transformations from the training data. `bake()` applies them to the test data; `new_data = NULL` returns the processed training data.

Machine learning in R

Fitting a model

The parsnip package provides a common syntax for defining models.

```
library(randomForest)
rf10 <- rand_forest(trees = 10, mode = "classification") |>
  set_engine("randomForest") |>
  fit(trump ~ ., data = data_training)
```

Machine learning in R

Making predictions: Training and test data

```
train_preds10 <- bind_cols(
  data_training, predict(rf10, data_training))
test_preds10 <- bind_cols(
  data_testing, predict(rf10, data_testing))
```

Precision, recall, and F1: Training versus test

```
class_metrics <- metric_set(precision, recall, f_meas)
train_metrics10 <- class_metrics(
  train_preds10, truth = trump, estimate = .pred_class,
  event_level = "second")
test_metrics10 <- class_metrics(
  test_preds10, truth = trump, estimate = .pred_class,
  event_level = "second")
```

`event_level = "second"` makes a Trump vote (1) the positive class.

Machine learning in R

Baseline: Random choice

```
coin <- data_testing |>
  mutate(.pred_class = factor(
    rbinom(n(), 1, 0.5), levels = c(0, 1)))
coin |>
  class_metrics(truth = trump, estimate = .pred_class,
                event_level = "second")
```

```
## # A tibble: 3 x 3
##   .metric   .estimator .estimate
##   <chr>     <chr>          <dbl>
## 1 precision binary         0.39
## 2 recall    binary         0.538
## 3 f_meas    binary         0.452
```

Machine learning in R

Predicted probabilities and AUC

Use `type = "prob"` to return predicted probabilities.

```
probs10 <- rf10 |>
  predict(data_testing, type = "prob") |>
  bind_cols(data_testing) |>
  select(trump, .pred_0, .pred_1)
```

Predicted probabilities and AUC

```
probs10 |> roc_auc(trump, .pred_1, event_level = "second") |>
  select(.metric, .estimate)
```

```
## # A tibble: 1 x 2
##   .metric .estimate
##   <chr>       <dbl>
## 1 roc_auc     0.765
```

Machine learning in R

Growing a forest

```
rf500 <- rand_forest(trees = 500, mode = "classification") |>
  set_engine("randomForest") |>
  fit(trump ~ ., data = data_training)
```

Machine learning in R

Precision, recall, and F1

```
train_preds500 <- bind_cols(
  data_training, predict(rf500, data_training))
test_preds500 <- bind_cols(
  data_testing, predict(rf500, data_testing))
```

Machine learning in R

Precision, recall, and F1

```
class_metrics(
  train_preds500, truth = trump, estimate = .pred_class,
  event_level = "second")
```

```
## # A tibble: 3 x 3
##   .metric   .estimator .estimate
##   <chr>     <chr>          <dbl>
## 1 precision binary         1
## 2 recall    binary         0.987
## 3 f_meas    binary         0.994
```

```
test_metrics500 <- class_metrics(
  test_preds500, truth = trump, estimate = .pred_class,
  event_level = "second")
```

Comparing the forests

```
test_comparison <- bind_rows(
  "10 trees" = test_metrics10,
  "500 trees" = test_metrics500, .id = "model")
test_comparison
```

```
## # A tibble: 6 x 4
##   model      .metric   .estimator .estimate
##   <chr>      <chr>     <chr>          <dbl>
## 1 10 trees  precision binary         0.638
## 2 10 trees  recall    binary         0.510
## 3 10 trees  f_meas    binary         0.567
## 4 500 trees precision binary         0.711
## 5 500 trees recall    binary         0.559
## 6 500 trees f_meas    binary         0.625
```

Machine learning in R

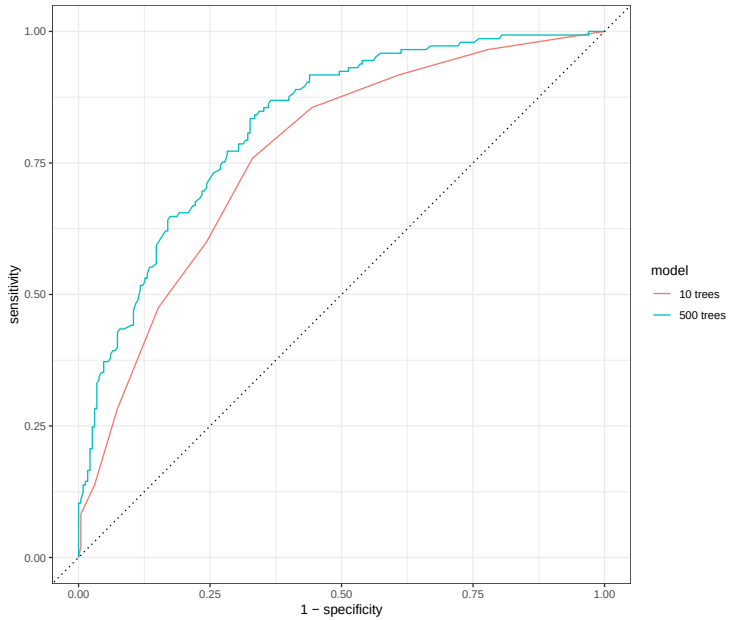
Model comparison

```
probs500 <- predict(rf500, data_testing, type = "prob") |>
  bind_cols(data_testing)
both <- bind_rows("10 trees" = probs10, "500 trees" = probs500,
                  .id = "model")
```

Machine learning in R

Model comparison

```
both |>
  group_by(model) |>
  roc_curve(trump, .pred_1, event_level = "second") |>
  autoplot()
```



Machine learning in R

Adding party identification and 2016 vote

```
party_recipe <- training(data_split) |>
  recipe(trump ~ .) |>
  step_dummy(all_factor_predictors()) |>
  prep()

party_training <- party_recipe |> bake(new_data = NULL)
party_testing <- party_recipe |> bake(testing(data_split))
```

Machine learning in R

Adding party identification and 2016 vote

```
rf_party <- rand_forest(trees = 500, mode = "classification") |>
  set_engine("randomForest") |>
  fit(trump ~ ., data = party_training)
```

Machine learning in R

Comparing all three models: Precision, recall, and F1

```
test_preds_party <- bind_cols(
  party_testing, predict(rf_party, party_testing))
test_metrics_party <- class_metrics(
  test_preds_party, truth = trump, estimate = .pred_class,
  event_level = "second")
all_metrics <- bind_rows(
  "10 trees" = test_metrics10, "500 trees" = test_metrics500,
  "500 trees + party ID and 2016 vote" = test_metrics_party,
  .id = "model")
```

Machine learning in R

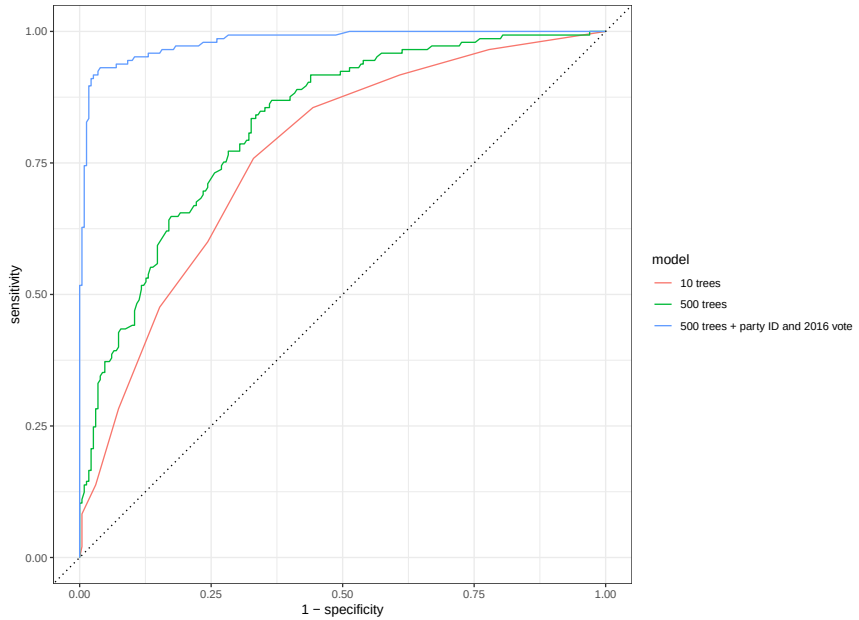
Comparing all three models

```
probs_party <- predict(rf_party, party_testing, type = "prob") |>
  bind_cols(party_testing)
all_models <- bind_rows(
  "10 trees" = probs10, "500 trees" = probs500,
  "500 trees + party ID and 2016 vote" = probs_party,
  .id = "model")
```

Machine learning in R

Comparing all three models: ROC

```
all_models |>
  group_by(model) |>
  roc_curve(trump, .pred_1, event_level = "second") |>
  autoplot()
```



Exercise

Can simple models make good predictions?

- Specify a model with only five features, excluding party ID and 2016 vote
- Fit a 500-tree forest using your features and report the out-of-sample AUC
- How does your model compare with the forest using all features?

Exercise

Selecting five features

```
# Replace feature1 to feature5 with your features
five_recipe <- training(data_split) |>
  recipe(trump ~ feature1 + feature2 + feature3 +
           feature4 + feature5) |>
  step_dummy(all_factor_predictors()) |>
  prep()
five_training <- bake(five_recipe, new_data = NULL)
five_testing <- bake(five_recipe, testing(data_split))
```

Exercise

Fitting the model

```
rf_five <- rand_forest(trees = 500, mode = "classification") |>
  set_engine("randomForest") |>
  fit(trump ~ ., data = five_training)
```

Exercise

Evaluating predictions

```
five_train_preds <- bind_cols(
  five_training, predict(rf_five, five_training))
five_test_preds <- bind_cols(
  five_testing, predict(rf_five, five_testing))
class_metrics(five_train_preds, truth = trump,
              estimate = .pred_class, event_level = "second")
```

Exercise

Comparing test predictions

```
five_test_metrics <- class_metrics(
  five_test_preds, truth = trump, estimate = .pred_class,
  event_level = "second")
bind_rows("Five features" = five_test_metrics,
          "All features" = test_metrics500, .id = "model")
predict(rf_five, five_testing, type = "prob") |>
  bind_cols(five_testing) |>
  roc_auc(trump, .pred_1, event_level = "second")
```

Summary

- Machine learning techniques allow us to "automate discovery from data"
- Supervised and unsupervised ML techniques
- Prediction vs. explanation
- Underfitting and overfitting
- Validation metrics
- Regularization
- Common algorithms

Next class

Supervised text classification (October 23)

- Classifying texts with supervised learning
- Cross-validation, hyperparameter searches, and model comparison
- Data quality and predictive performance